Национальный исследовательский университет ИТМО
Факультет информационных технологий и программирования
Прикладная математика и информатика

Методы оптимизации

Отчет по лабораторной работе №4

Работу выполнили:

Коротков Арсентий M32351

Рябов Вадим M32351

Хайруллин Александр M32351

Преподаватель:

Ким Станислав Евгеньевич

г. Санкт-Петербург

2023 г.

Задание 1.

Теория

Если минимизируемая функция имеет вид: $f = \sum_{i=1}^{n} f_i$, то для ее минимизации можно воспользоваться стохастическим градиентным спуском, основная идея которого заключается в том, что мы будем вычислять градиент лишь в одной из n компонент в каком-то случайном порядке. Это увеличит количество шагов, но уменьшит ресурсы, тратящиеся на вычисление одного шага, так как градиент берется лишь в одной компоненте.

Чтобы гарантированно посчитать градиент для всех компонент из суммы, можно считать его последовательно. С целью нахождения оптимального потребления ресурсов и числа шагов, возможно изменить данный алгоритм, введя такое понятие как батч, который отвечает за то, на скольких компонентах мы берем градиент за одну итерацию.

$$\sqsupset g = \sum_{j} \nabla f_j$$

Стохастический градиентный спуск и его модификации:

Название	Формулы	Принцип работы
SGD	$w_{k+1} = w_k - \alpha g(x_k)$	Указан выше.
Momentum	Классический вид: $v_{k+1} = \gamma v_k + (1 - \gamma) g(x_k)$ $w_{k+1} = w_k - \alpha v_{k+1}$ Альтернативный: $v_{k+1} = \gamma_k v_k - \alpha_k g(x_k)$	Смысл работы данной модификации проще понять, проведя аналогию с инерцией в реальной жизни. Некоторый объект, имея накопленный импульс и двигаясь по какой-то местности по

	$w_{k+1} = w_k + v_{k+1}$	инерции, не остановит свое движение из-за небольших неровностей на маршруте его траектории.
Nesterov	Классический вид: $v_{k+1} = \gamma v_k + (1 - \gamma) g(w_k - \alpha\gamma v_k)$ $w_{k+1} = w_k - \alpha v_{k+1}$ Альтернативный: $v_{k+1} = \gamma_k v_k - \alpha_k g(w_k + \gamma_k v_k)$ $w_{k+1} = w_k + v_{k+1}$	Алгоритм Нестерова улучшает моментум, считая градиент не в текущей точке, а в точке, где бы мы оказались, следуя мы импульсу нашей инерции. С помощью этого, наши колебания в направлении движения уменьшаются. В мире выпуклых функций алгоритм считается не улучшаемым.
Adagrad	$G_{k+1} = G_k + (g(w_k))^2$ $w_{k+1} = w_k - \frac{\alpha}{\sqrt{G_{k+1} + \varepsilon}} g(w_k)$	АдаГрад предполагает динамически подбирать размер шага для каждой константы по отдельности. Однако, нам нельзя слишком уменьшать размер шага, потому что выйди мы на плато, мы рискуем там и застрять. Потому такие девиации мы стараемся компенсировать.
RMSProp	$G_{k+1} = \gamma G_k + (1 - \gamma)(g(w_k))^2$ $w_{k+1} = w_k - \frac{\alpha}{\sqrt{G_{k+1} + \varepsilon}} g(w_k)$	Берется предыдущий алгоритм, однако нормы градиентов не просто складываются, но усредняются в скользящем режиме. Данные манипуляции позволяют замедлить уменьшение размера шага и всё ещё учитывать историю градиентов.
Adam	$v_{k+1} = \beta_1 v_k + (1 - \beta_1) g(w_k)$ $G_{k+1} = \beta_2 G_k + (1 - \beta_2)(g(w_k))^2$	Данный алгоритм объединяет в себе идеи двух основных идей, описываемых в

	$w_{k+1} = w_k - \frac{\alpha}{\sqrt{G_{k+1}+\varepsilon}} v_{k+1}$	алгоритмах выше (Adam ≡ Adaptive Momentum)

Также можно ввести коррекцию:

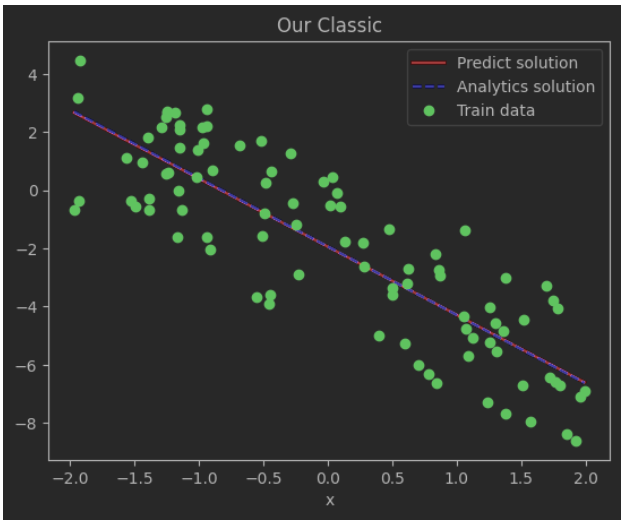
$$\tilde{v}_{k+1} = \frac{v_{k+1}}{1-\beta_1^{k+1}}$$

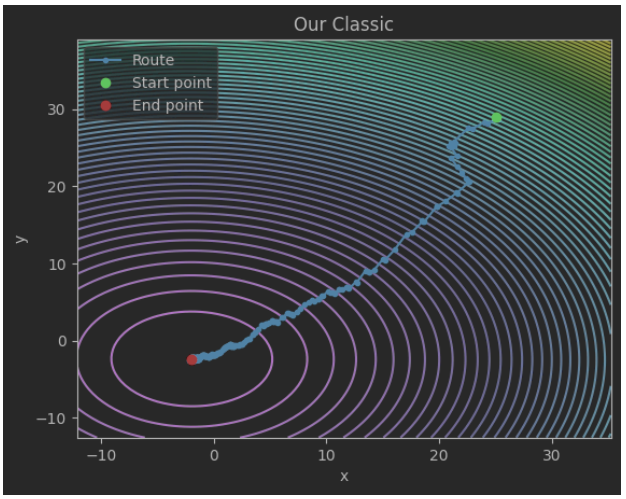
$$\tilde{G}_{k+1} = \frac{G_{k+1}}{1-\beta_2^{k+1}}$$

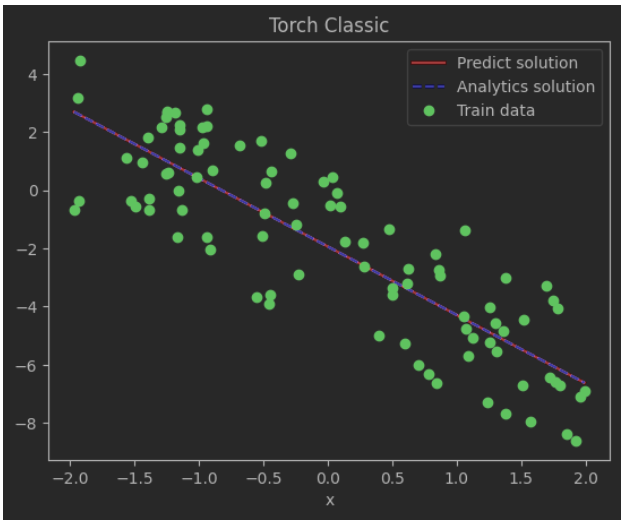
И подставлять в рекуррентную формулу для весов скорректированные величины соответственно. Так как $G_1 = v_1 = 0$, то параметры без коррекции будут слишком сильно от g_1, соответственно коррекция решает эту проблему. Без коррекции же значения нормализуются сами собой через некоторое количество итераций.

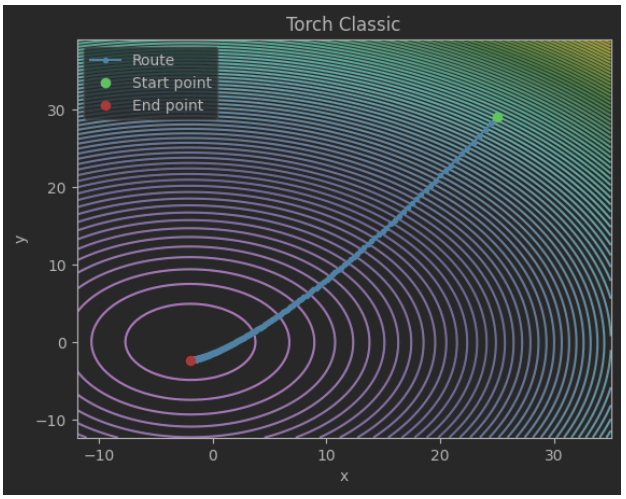
Сравнения эффективности собственной реализации алгоритмов и из torch.optim

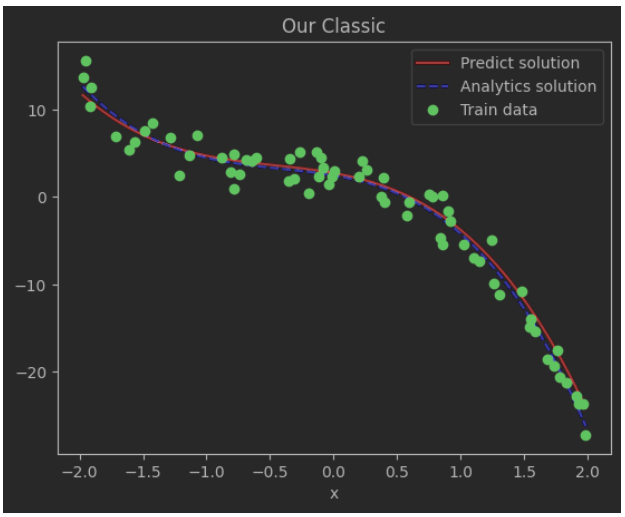
SGD

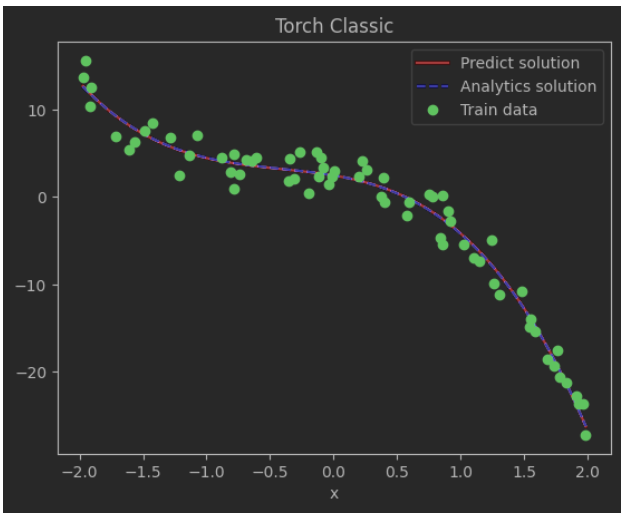

Our Classic
Predict solution
Analytics solution
Train data
x


Our Classic
Route
Start point
End point
x
y

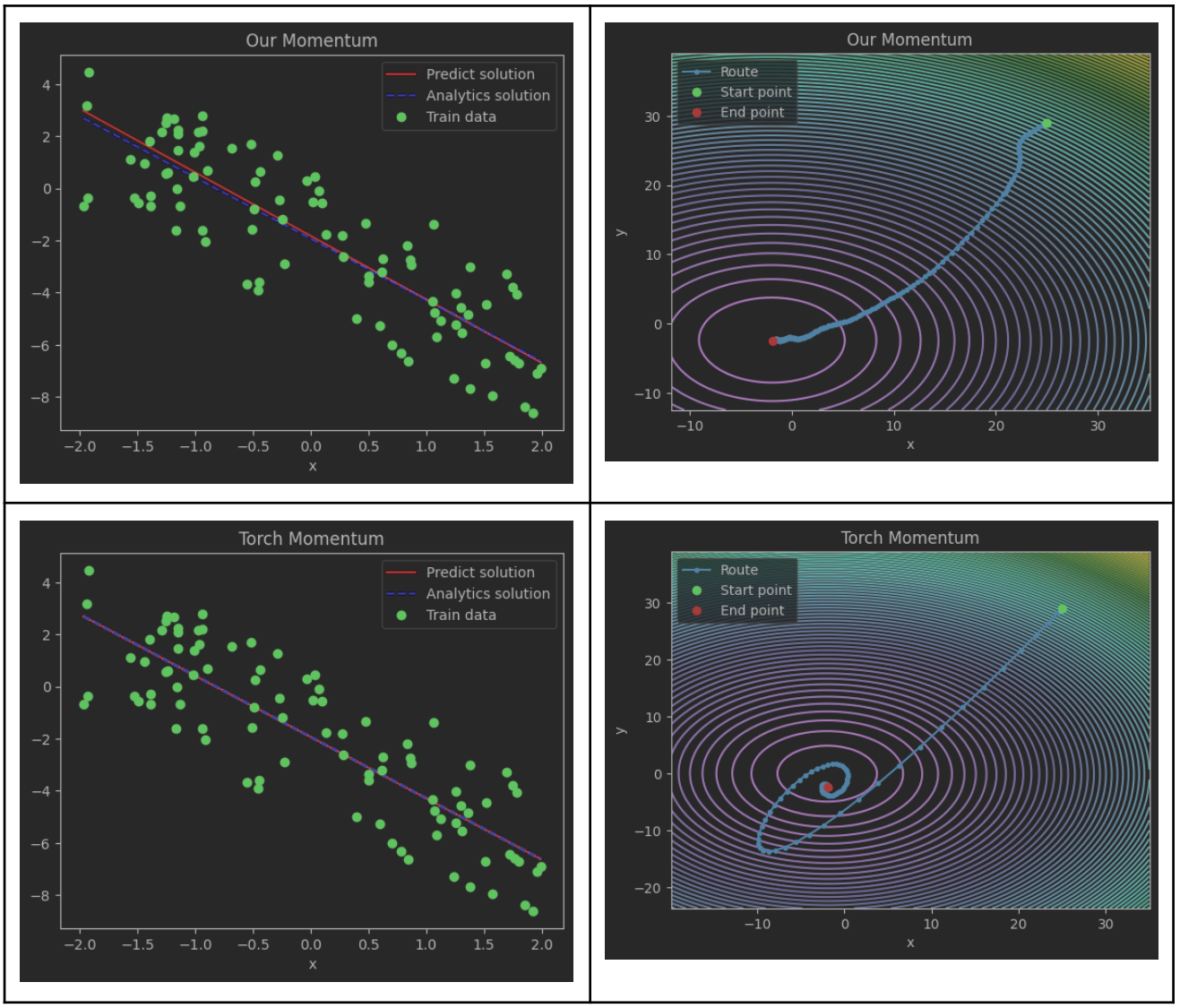

Torch Classic
Predict solution
Analytics solution
Train data
x

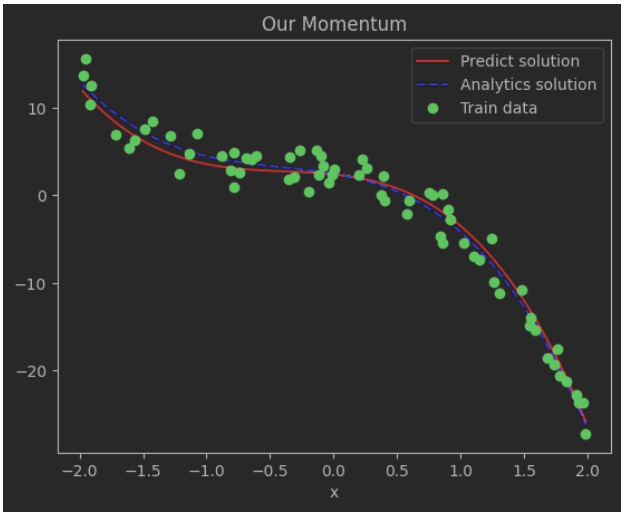

Torch Classic
Route
Start point
End point
x
y

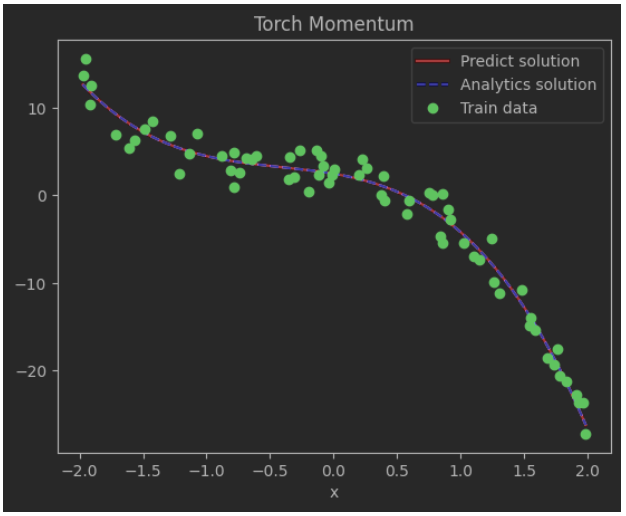

Our Classic
Predict solution
Analytics solution
Train data
x


Torch Classic
Predict solution
Analytics solution
Train data
x

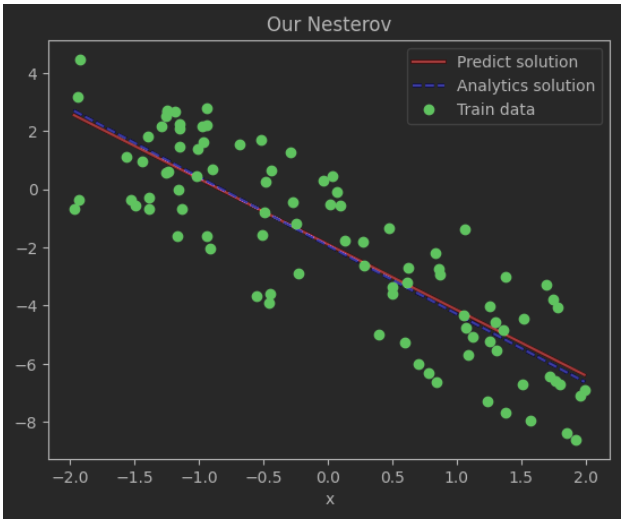
Momentum

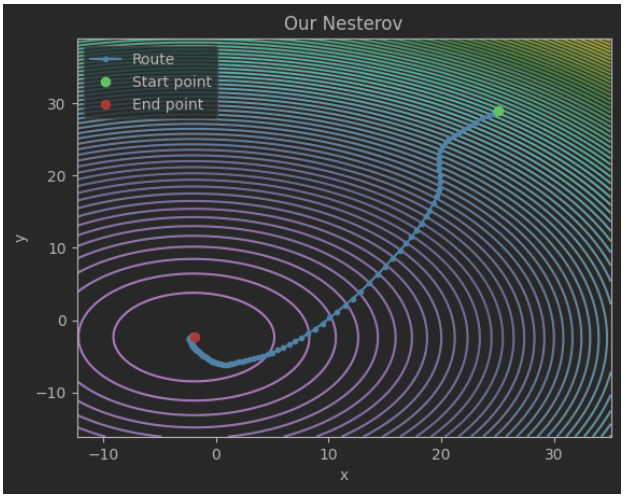

Our Momentum
Predict solution
Analytics solution
Train data
x
Our Momentum
Route
Start point
End point
x
y
Torch Momentum
Predict solution
Analytics solution
Train data
x
Torch Momentum
Route
Start point
End point
x
y

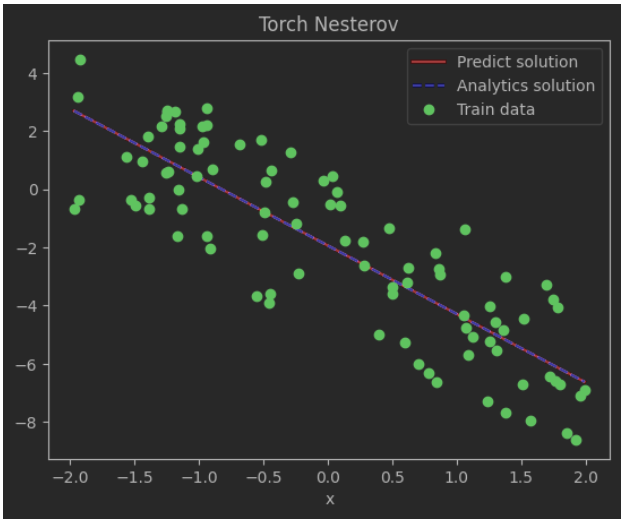

Our Momentum
Predict solution
Analytics solution
Train data
x

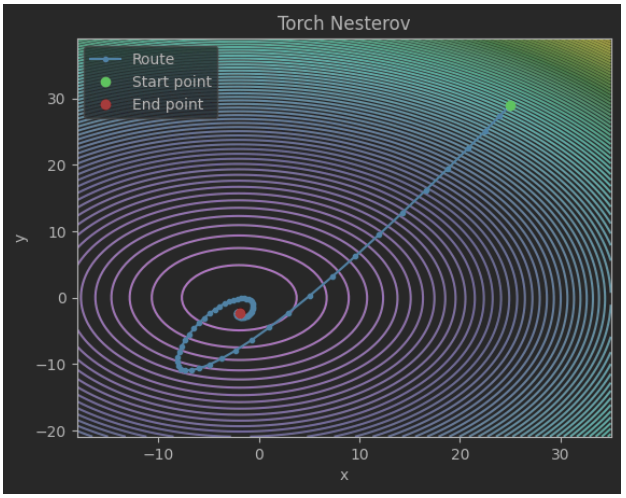

Torch Momentum
Predict solution
Analytics solution
Train data
x

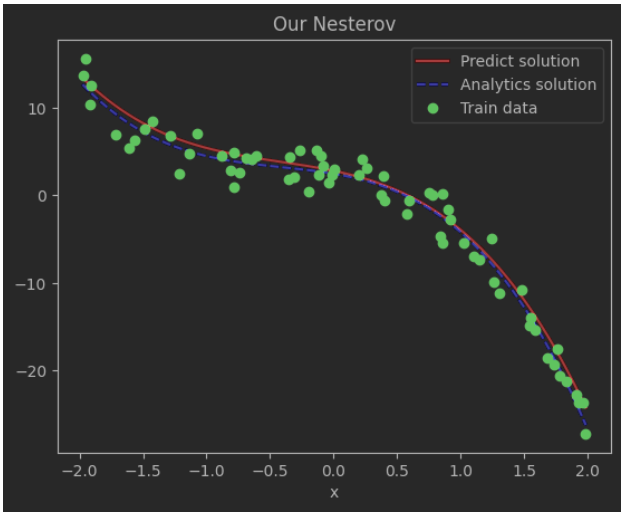
Nesterov

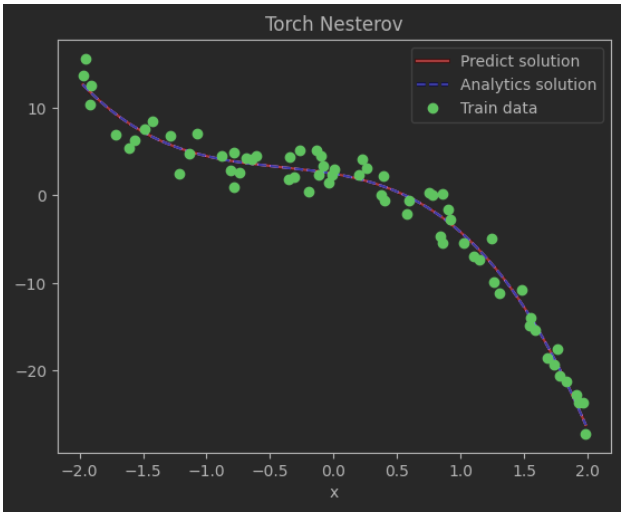

Our Nesterov
Predict solution
Analytics solution
Train data
x


Our Nesterov
Route
Start point
End point
x
y

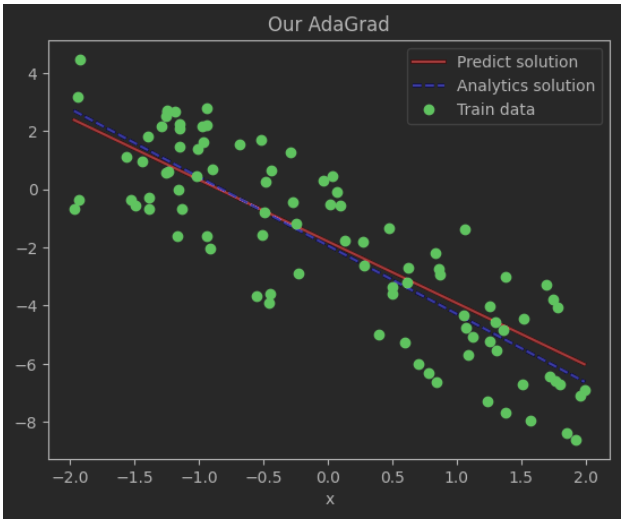

Torch Nesterov
Predict solution
Analytics solution
Train data
x

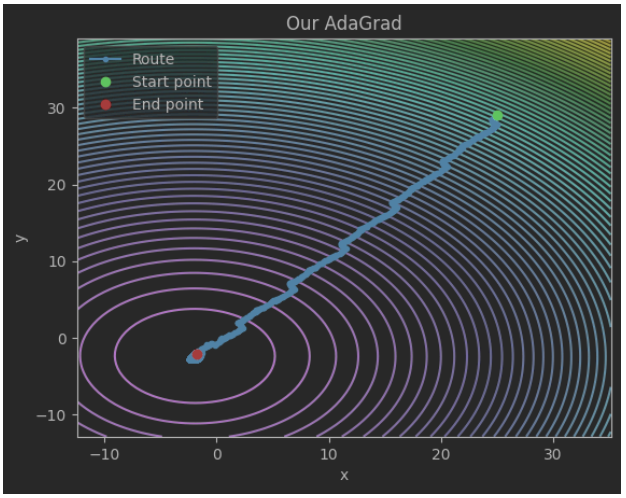

Torch Nesterov
Route
Start point
End point
x
y

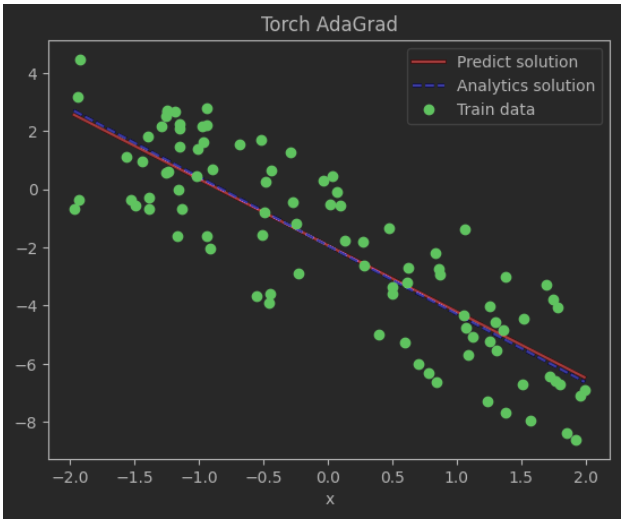

Our Nesterov
Predict solution
Analytics solution
Train data
x

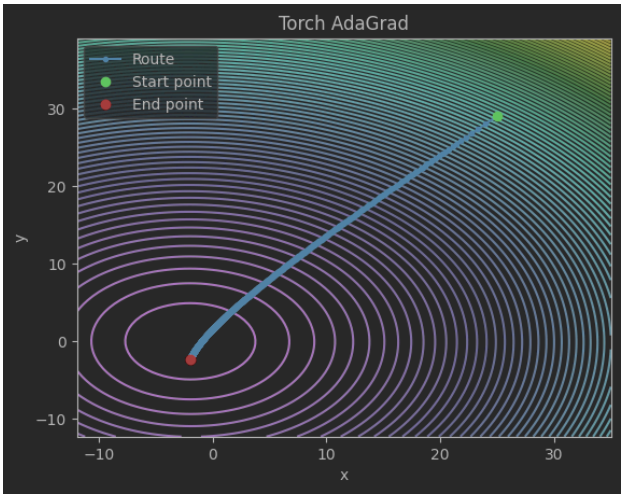

Torch Nesterov
Predict solution
Analytics solution
Train data
x

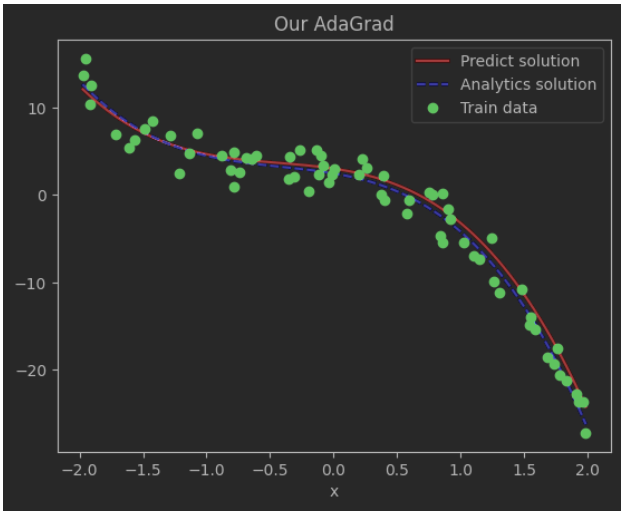
AdaGrad

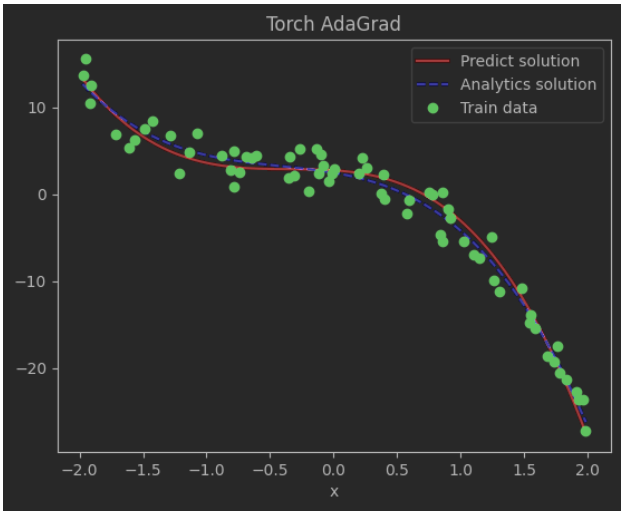

Our AdaGrad
Predict solution
Analytics solution
Train data
x


Our AdaGrad
Route
Start point
End point
x
y

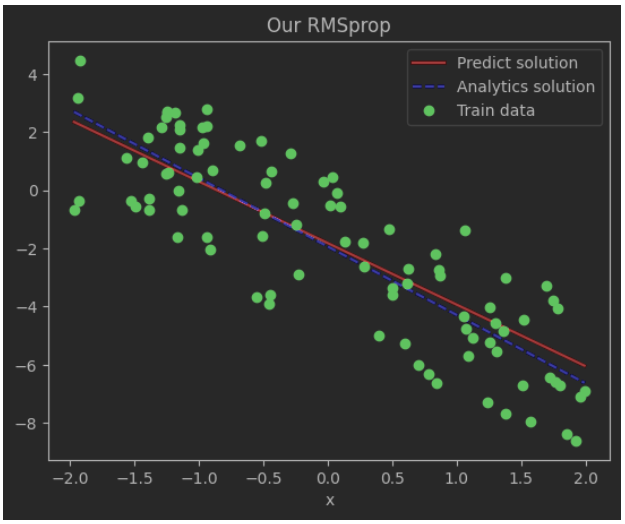

Torch AdaGrad
Predict solution
Analytics solution
Train data
x

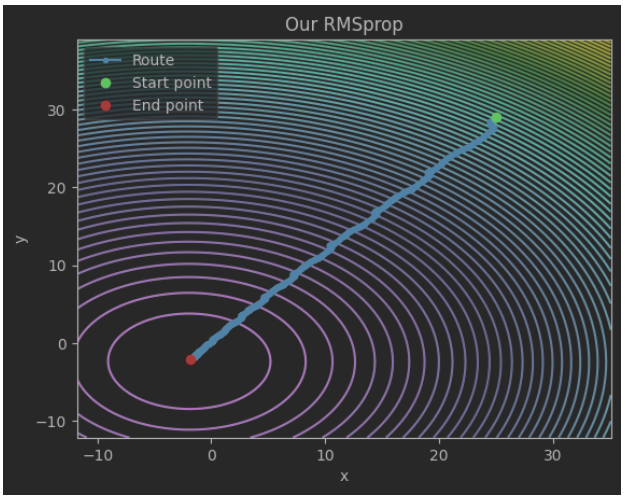

Torch AdaGrad
Route
Start point
End point
x
y

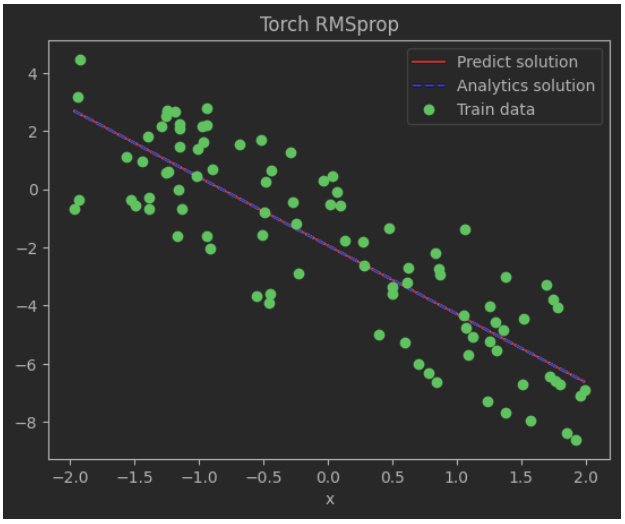

Our AdaGrad
Predict solution
Analytics solution
Train data
x

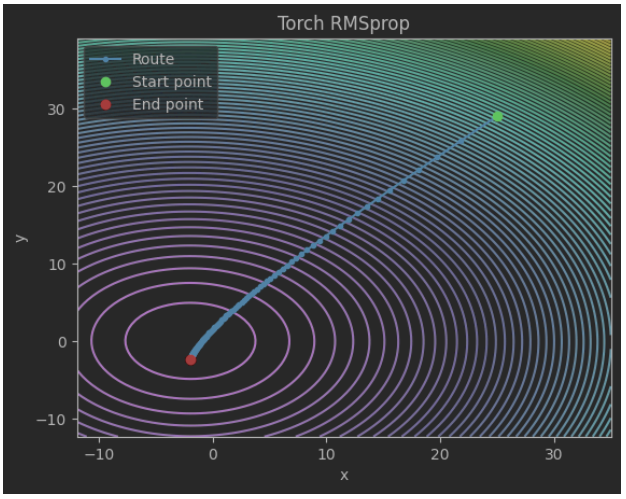

Torch AdaGrad
Predict solution
Analytics solution
Train data
x

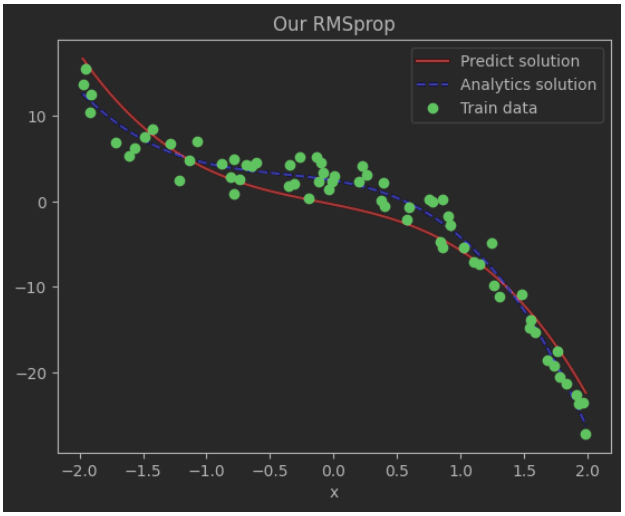
RMSProp

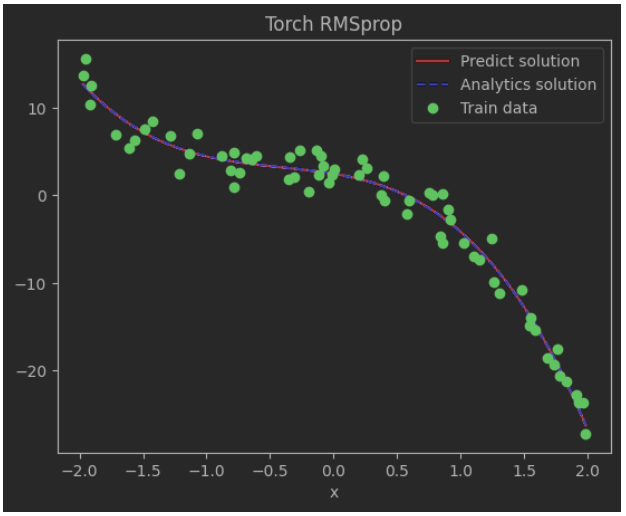

Our RMSprop
Predict solution
Analytics solution
Train data
x


Our RMSprop
Route
Start point
End point
x
y

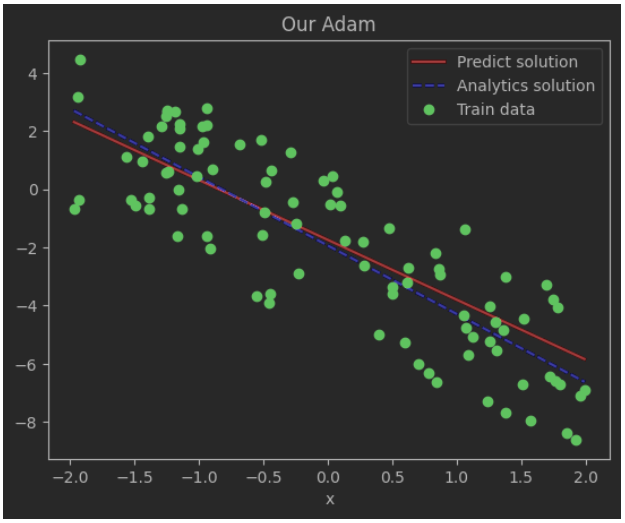

Torch RMSprop
Predict solution
Analytics solution
Train data
x

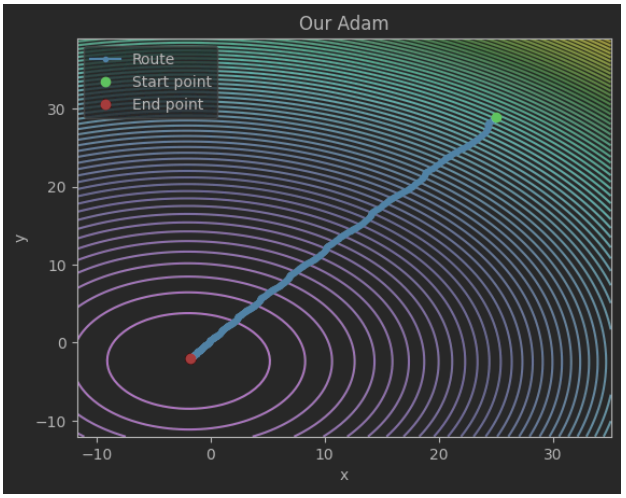

Torch RMSprop
Route
Start point
End point
x
y

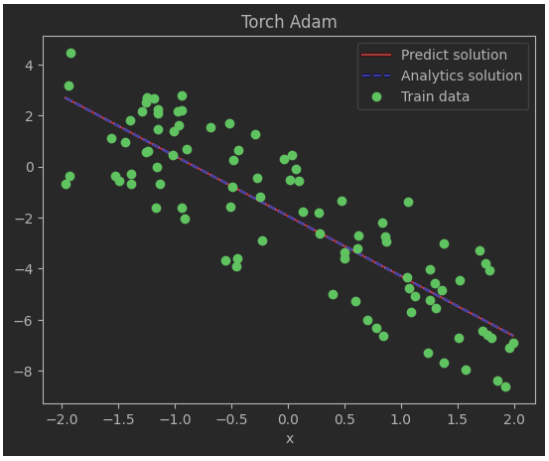

Our RMSprop
Predict solution
Analytics solution
Train data
10
0
−10
−20
−2.0
−1.5
−1.0
−0.5
0.0
0.5
1.0
1.5
2.0
x

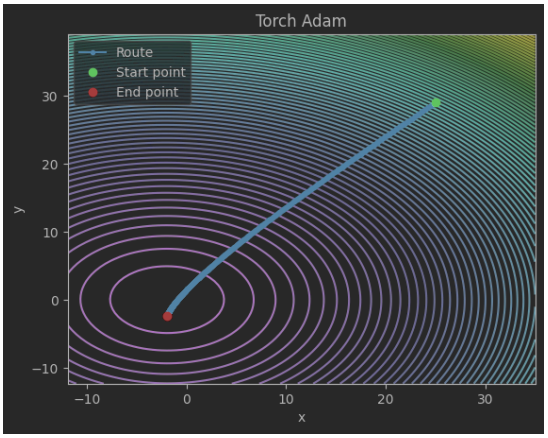

Torch RMSprop
Predict solution
Analytics solution
Train data
10
0
−10
−20
−2.0
−1.5
−1.0
−0.5
0.0
0.5
1.0
1.5
2.0
x

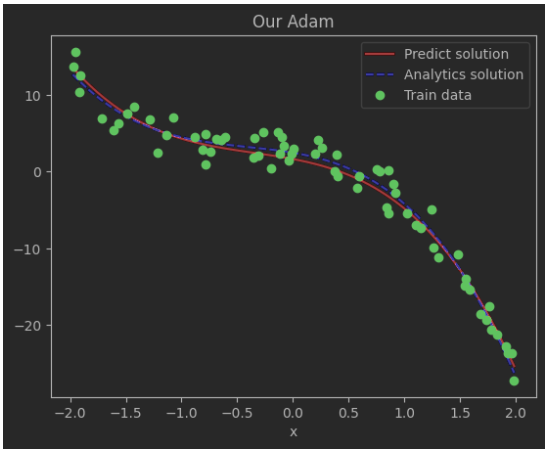
Adam

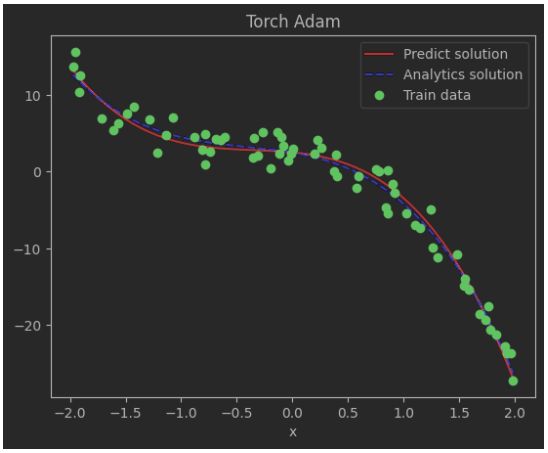

Our Adam
Predict solution
Analytics solution
Train data
4
2
0
−2
−4
−6
−8
−2.0
−1.5
−1.0
−0.5
0.0
0.5
1.0
1.5
2.0
x


Our Adam
Route
Start point
End point
30
20
10
0
−10
y
−10
0
10
20
30
x









Метрики:

Method	SGD	Momentum	Nesterov	AdaGrad	RMSProp	Adam
Наши реализации						
Time	2.526271939	0.353824377	3.933111072	4.465481281	0.975912213	1.469088793
Memory	3864	3864	3864	3864	3984	4104
Relative error	0.120193703	0.087370708	0.137815961	0.186010414	1.030159716	0.102774386

PyTorch						
Time	0.569474697	0.571221948	0.567816973	0.680977941	0.51624465	0.596152067
Memory	3094	2927.5	2959.5	3215.5	3275.5	4249.5
Relative error	9.71528E-05	0	0	0.106991349	0.000172051	0.04960397

<u>Задание 2.</u>

2.a

Гаусс - Ньютон

Если минимизируемая функция имеет вид $f(x) = \frac{1}{2}\sum_{i=1}^{m} r_i^2(x)$, то для ее минимизации можно воспользоваться итеративным алгоритмом Гаусса-Ньютона. Он основывается на алгоритме Ньютона, и в отличии от него минимизирует только сумму квадратов, однако не требует вычисления второй производной, что может быть затратно при сложных r_i и большом m.

<u>Описание работы метода:</u>

Необходимым условием для работы алгоритма является обратимость матрицы Якоби на каждом шаге. В противном случае пересчет значения точки будет невозможным ввиду следующих формул:

$R(X) = (r_1(X), \ldots, r_m(X))^T$

$R_k = R(X_k)$ - вектор значений функций r_i на k-й итерации

$\mathrm{J}(X) = [\frac{\partial r_i}{\partial x_j}(X)]$

$\mathrm{J}_k = \mathrm{J}(X_k)$ - матрица Якоби для функций r_i на k-й итерации

$X_{k+1} = X_k - (\mathrm{J}_k^T \mathrm{J}_k)^{-1} \mathrm{J}_k^T R_k$ - рекуррентная формула для вычисления k+1-го приближения минимума

Powell's DogLeg

DogLeg относится к семейству trust region методов. В общем случае они работают на каждом шаге следующим образом: есть некоторый доверительный регион, в котором квадратичная модель аппроксимирует функцию, далее либо выбирается точка, принадлежащая доверительному региону, как следующее приближение минимума (возможно, с расширением региона), либо происходит сужение региона.

<u>Описание работы метода:</u>

B_k - матрица Гессе на k-й итерации

g_k - градиент функции на k-й итерации

$m_k(p) = f_k + g_k^T p + \frac{1}{2} p^T B_k p$ - модель функции на k-й итерации

$\rho_k = \frac{f(X_k) - f(X_k + p_k)}{m_k(0) - m_k(p)}$ - коэффициент соответствия модели на k-й итерации

Δ_k - радиус доверительного региона на k-й итерации

$X_{k+1} = X_k + p_k$ - рекуррентная формула для вычисления k+1-го приближения минимума

Шаг вычисляется следующим образом. $p_k^b = - B_k^{-1} g_k$ - решение для модели без учета ограничений. Если решение принадлежит доверительному региону ($||p_k^b|| \leq \Delta_k$), то оно выбирается в качестве шага. В противном случае, вычисляется $p_k^u = - \frac{g_k^T g_k}{g_k^T B_k g_k} g$ - решение для модели по направлению антиградиента без учета ограничений. Если решение не принадлежит доверительному региону, то в качестве шага выбирается вектор, сонаправленный антиградиенту, длины радиуса доверительного региона. Если же лежит, то исходя из уравнения $||p_k^u + (p_k^b - p_k^u)\alpha|| = \Delta_k$ вычисляется коэффициент, и в качестве шага выбирается вектор $p_k^u + (p_k^b - p_k^u)\alpha$.

Такие шаги обусловлены тем, что при увеличении радиуса доверительного региона, минимум для модель в рамках ограничений движется по кривой, вписанной в угол между p_k^u и $p_k^b - p_k^u$.

Для наглядности приведем рисунок:

Можно заметить, что p^u и $p^b - p^u$ образуют угол, похожий на собачью ногу. p^b

Пересчет происходит следующим образом. Если коэффициент соответствия низкий $\rho_k < \frac{1}{4}$, то аппроксимация признается неразумной и доверительный регион сужается: $\Delta_{k+1} = \frac{1}{4}\Delta_k$. В противном случае, если коэффициент соответствия высокий $\rho_k > \frac{3}{4}$ и длина шага равна радиусу доверительного региона (с точностью eps) $||p_k|| = \Delta_k$, то $\Delta_{k+1} = min(2\Delta_k, \Delta_{max})$, где Δ_{max} - максимально допустимый радиус доверительного региона. Если эти условия не выполняются, то радиус допустимого региона не изменяется.

BFGS

Метод BFGS относится к итеративным квазиньютоновским методам. В квазиньютоновских методах не вычисляется напрямую матрица Гессе функции. Вместо этого она оценивается приближенно, исходя из сделанных до этого шагов. Данный метод находит минимум любой дважды непрерывно дифференцируемой выпуклой функции. Несмотря на эти теоретические ограничения, как показывает опыт, BFGS хорошо справляется и с невыпуклыми функциями.

<u>Описание работы метода:</u>

$y_k = \nabla f_{k+1} - \nabla f_k$

$s_k = X_{k+1} - X_k$

$\rho_k = \frac{1}{y_k^T s_k}$

$H_{k+1} = (I - \rho_k s_k y_k^T) H_k (I - \rho_k y_k s_k^T) + \rho_k s_k s_k^T$ - рекуррентная формула для вычисления k+1-го приближения к обратной к матрице Гесса

$p_k = - H_k \nabla f_k$ - формула шага на k-й итерации

$X_{k+1} = X_k + \alpha_k p_k$ - рекуррентная формула для вычисления k+1-го приближения минимума, где α_k вычисляется с помощью одномерного поиска с условиями Вольфе

Минус метода в том, что требуется симметричность и положительная определенность H_k, но при выполнении итераций строго по формулам и поиска с Условиями Вольфе, эти условия соблюдаются.

LBFGS

Метод LBFGS относится к итеративным квазиньютоновским методам. Является модификацией BFGS с оптимизацией размера используемой памяти (отсюда и L - Limited Memory). Предназначен для решения нелинейных задач с большим количеством неизвестных.

Как и оригинальный BFGS, LBFGS использует оценку обратной матрицы Гессе, но не хранит всю матрицу, а только несколько векторов, которые представляют приближение неявно. LBFGS поддерживает историю прошлых m (обычно, $1 \leq m \leq 30$) обновлений положения приближения и градиента (s_k и y_k). Эти обновления используются в алгоритме подсчета приближения обратной к матрице Гессе, основанном на рекуррентной формуле из BFGS, указанной в описании работы метода.

<u>Описание работы метода:</u>

$y_k = \nabla f_{k+1} - \nabla f_k$

$s_k = X_{k+1} - X_k$

$\rho_k = \frac{1}{y_k^T s_k}$

$V_k = I - \rho_k y_k s_k^T$

$\gamma_k = \frac{s_{k-1}^T y_{k-1}^T}{y_{k-1}^T y_{k-1}^T}$

$H_k^0 = \gamma_k I$ - начальное приближение обратной матрицы Гессе на k-й итерации

$H_k = (V_{k-1}^T \ldots V_{k-m}^T) H_k^0 (V_{k-m} \ldots V_{k-1}) +$

$+ \rho_{k-m} (V_{k-1}^T \ldots V_{k-m+1}^T) s_{k-m} s_{k-m}^T (V_{k-m+1} \ldots V_{k-1}) + \ldots +$

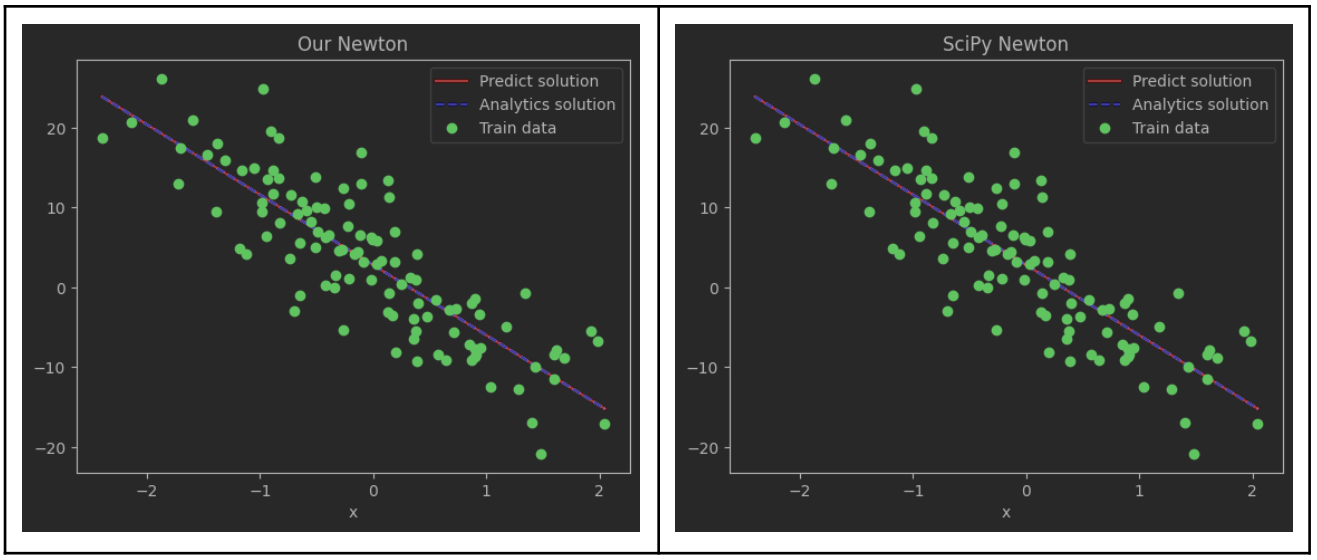
$+ \rho_{k-1} s_{k-1} s_{k-1}^T$ - формула для вычисления k-го приближения к обратной к матрице Гесса

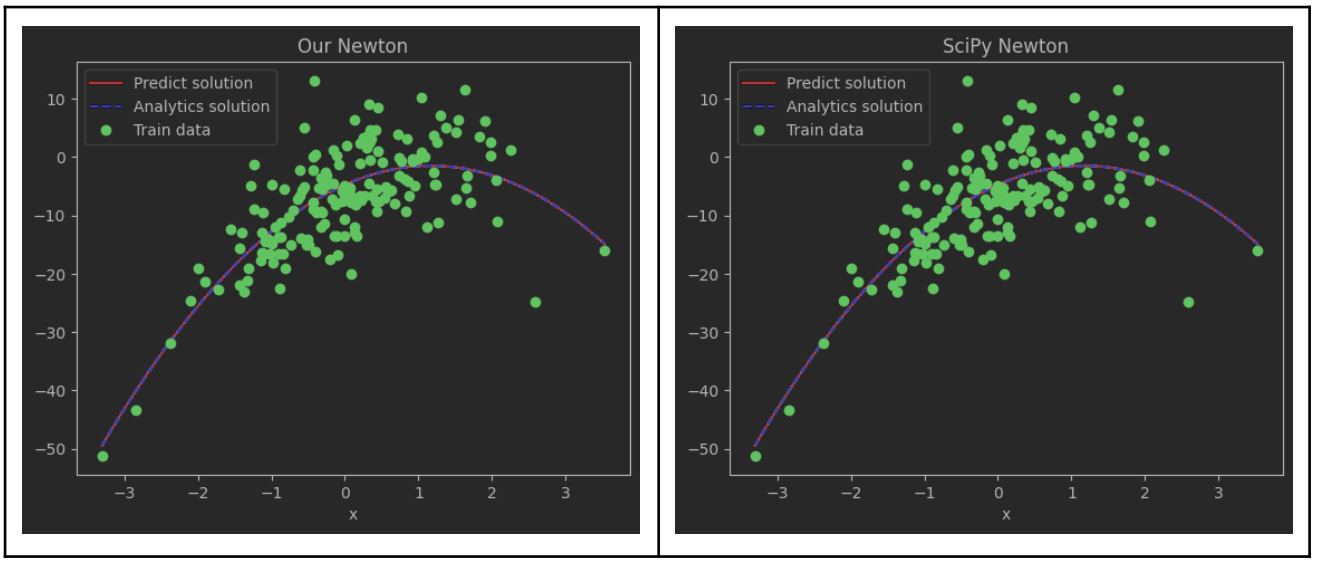
$p_k = -H_k \nabla f_k$ - формула шага на k-й итерации

$X_{k+1} = X_k + \alpha_k p_k$ - рекуррентная формула для вычисления k+1-го приближения минимума, где α_k вычисляется с помощью одномерного поиска с условиями Вольфе

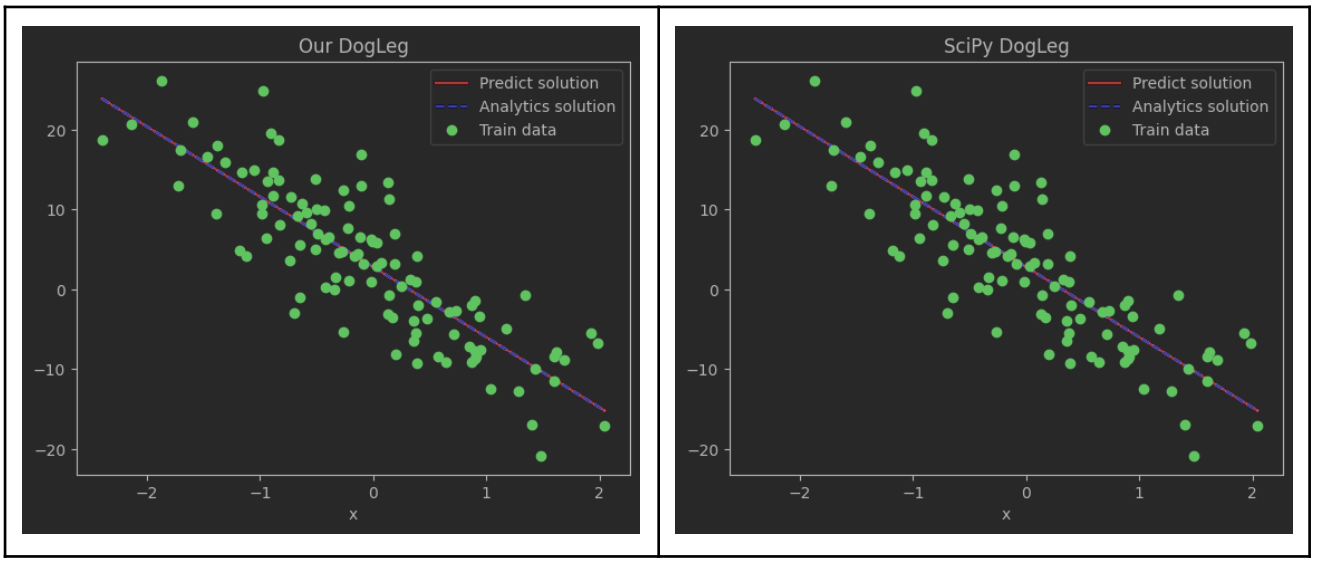
Сравнения эффективности собственной реализации алгоритмов и из scipy.optimize.minimize, scipy.optimize.least_squares:

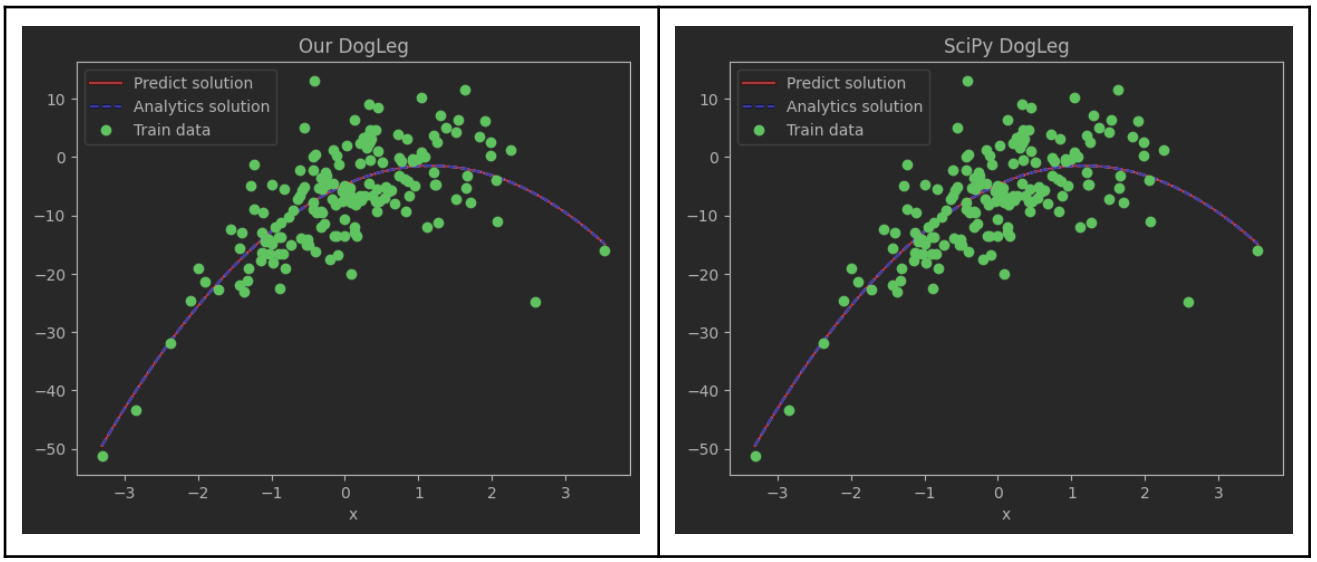
Gauss-Newton



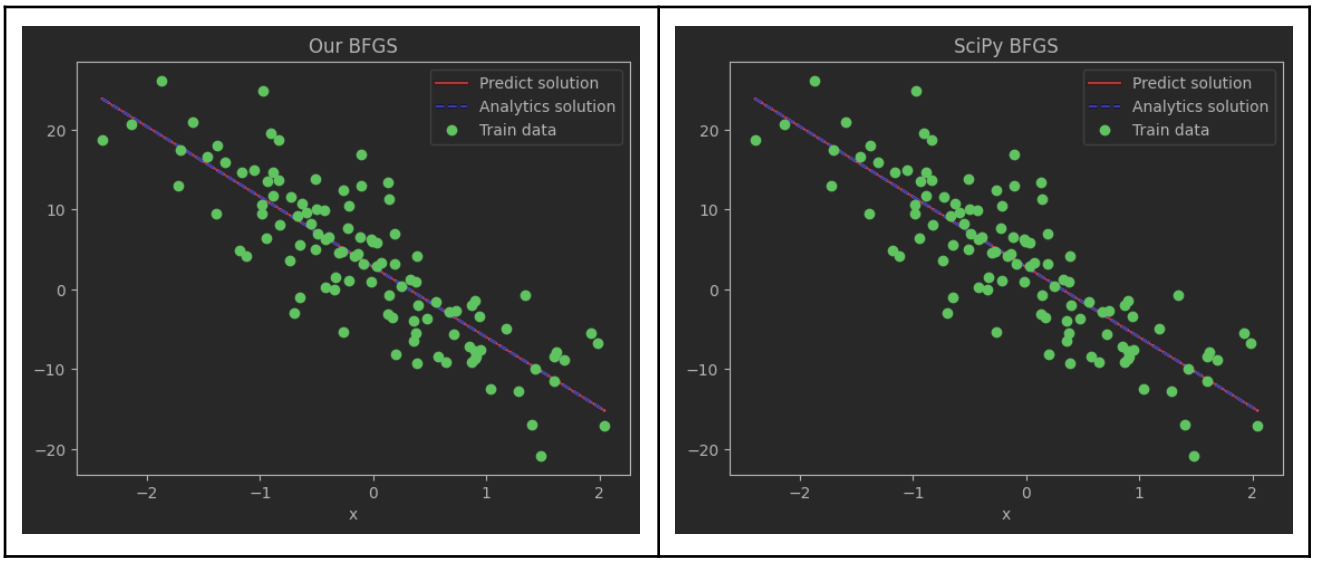


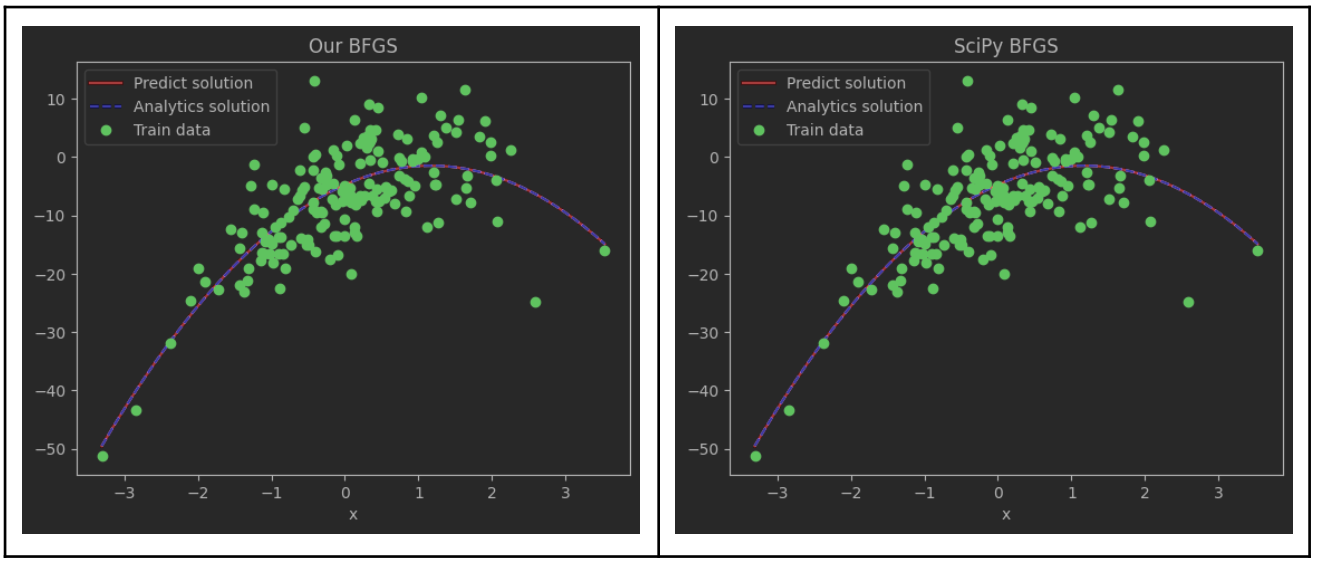
Powell's DogLeg



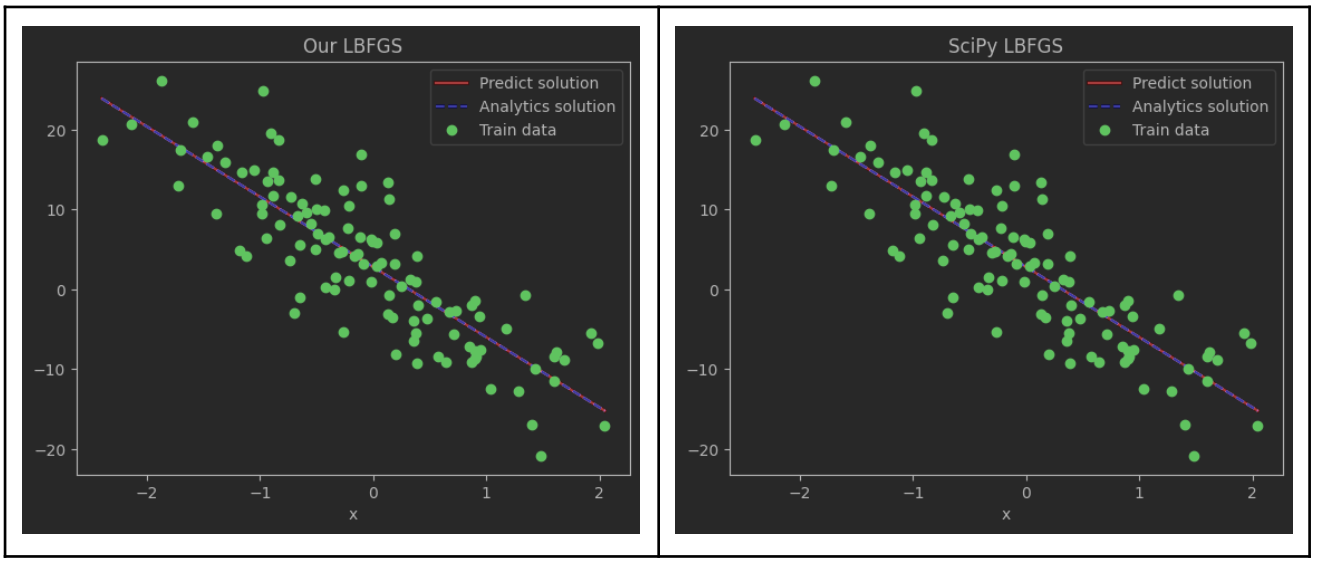

Our DogLeg
Predict solution
Analytics solution
Train data
x
SciPy DogLeg
Predict solution
Analytics solution
Train data
x

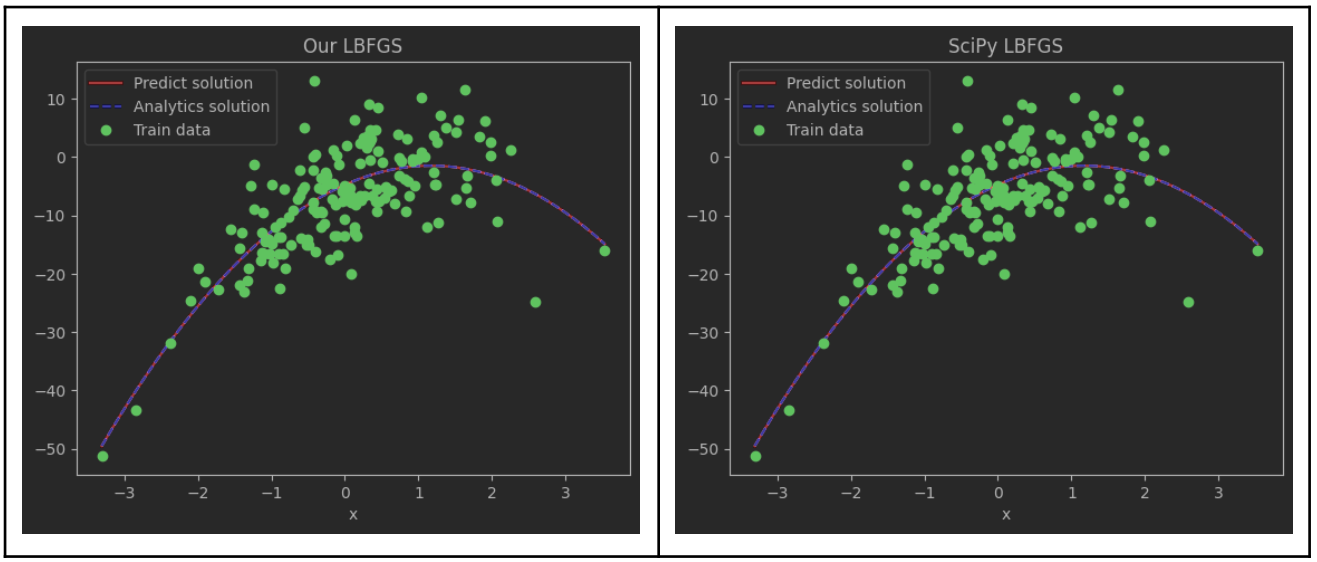
BFGS


Our BFGS
Predict solution
Analytics solution
Train data
x
SciPy BFGS
Predict solution
Analytics solution
Train data
x


Our BFGS
Predict solution
Analytics solution
Train data
x
SciPy BFGS
Predict solution
Analytics solution
Train data
x

LBFGS


Our LBFGS
Predict solution
Analytics solution
Train data
x
SciPy LBFGS
Predict solution
Analytics solution
Train data
x



Метрики:

Method	Gauss-Newton	Powell's DogLeg	BFGS	LBFGS
Наши реализации				
Time	1.253865123	10.88131547	0.007906437	0.000999331
Memory	26713	4904	4178.5	3877
Relative error	0.00061051838	0.00061054399	0.00061054398	0.00061054398
SciPy				
Time	0.024590254	0.020556211	0.001810789	0.002002001
Memory	362963.5	4379	5959	2998
Relative error	0.00061055082	0.00061054381	0.00061054399	0.00061054398

2.b

Была проведена серия из 100 экспериментов, ниже можете увидеть усредненные метрики:

Method grad calc	Numerical	Analytical	Torch
Time	0.71672473	0.914551797	0.114006791
Memory	6868.62	6422.78	6320.44
Relative error	0.000323803	0.045331478	0.000324106

2.c

Nelder-Mead

	No bounds	Bounds
Time	0.257874648	0.213922501
Memory	7604.666667	8413.666667
Relative error	1.048467407	1.871649921

L-BFGS-B

	No bounds	Bounds
Time	0.431667328	0.023090919
Memory	21522	21641
Relative error	0.000729887	1.871634346

Trust-Constr

	No bounds	Bounds
Time	0.618180434	0.220746597
Memory	20247	36003.6
Relative error	0.00065169	1.871634456

Дополнительное задание:

scipy.optimize.LinearConstraint

	On border	Inner
Time	0.832458253	1.042449975
Memory	631243.82	636501.78
Relative error	0.000746495	0.0223241592

scipy.optimize.NonlinearConstraint

	On border	Inner
Time	7.285788258	8.498876333
Memory	23772.66	23754.51
Relative error	0.0765370883	0.0689755963